

INCA Capacity-Driven Continual Learning: Full Implementation Plan, Baselines, and Datasets

Executive Summary

This document outlines a full implementation concept for an INCA-style continual learning system positioned against LLaMA Pro as the primary architecture-growth baseline. LLaMA Pro establishes that vertical Transformer block expansion can improve a pretrained language model by tuning only newly added blocks on a new corpus, particularly code and math data. [cite:4] The proposed INCA direction extends that idea by turning architectural growth into a control problem: the model should decide when to allocate more capacity, how to retrieve from accumulated blocks, and how to preserve previously learned domains without destructive forgetting.[cite:4]

The recommended paper framing is **capacity-aware continual learning with backward transfer as a central metric**. The main claim should be that fixed growth is a useful baseline, but adaptive growth is better because it grows only when the current trainable block is saturated, preserves older knowledge structurally, and can support better backward transfer under the same compute budget.[cite:4] This framing keeps the work in continual learning rather than pure post-pretraining, while avoiding dependence on a purely temporal-news narrative. [cite:65][cite:66]

Research Framing

Core problem statement

The target research question is: **Can a language model autonomously allocate additional architectural capacity only when its current capacity is exhausted, while preserving previously learned domains and maintaining or improving backward transfer?** This problem setting is motivated by the limitations of static architecture growth, where block addition is externally scheduled rather than inferred from learning dynamics.[cite:4]

LLaMA Pro is the correct baseline because it validates vertical expansion as a viable mechanism for knowledge injection, but it still performs growth as a fixed, human-specified design choice rather than an endogenous decision process.[cite:4][cite:25] INCA should therefore be positioned not as a rejection of LLaMA Pro, but as its next step: from **static block expansion** to **intelligent capacity allocation**.[cite:4]

Central thesis

The clean thesis for the paper is: **vertical expansion is useful, but the timing, routing, and consolidation policy should be learned from the model's own saturation signals rather than fixed in advance.**[cite:4] This preserves the importance of continual learning metrics such as average accuracy, backward transfer, and forward transfer, while letting the method stand on a capacity-driven foundation.[cite:65]

Conceptual comparison

Dimension	LLaMA Pro	INCA direction in this paper
Growth policy	Fixed block expansion decided by the experimenter.[cite:4]	Saturation-driven growth decided by model-internal signals.
Initialization	Identity/zero-style initialization to preserve base behavior at insertion.[cite:4]	Warm-start from the current trainable block to retain recent representational structure.
Retrieval	Implicit sequential stack.[cite:4]	Explicit selector-based retrieval across frozen and current blocks.
Continual learning story	Domain injection with preservation of prior ability.[cite:4]	Capacity-aware continual learning with retention and backward transfer.
Main novelty	Demonstrates viability of block expansion. [cite:4]	Turns expansion into a control system over growth, routing, and consolidation.

Full implementation idea

1. Base model choice

The implementation should begin with a **small or medium open language model** rather than a full LLaMA-scale reproduction, because the paper's novelty lies in adaptive capacity allocation rather than raw scale. The model should still support causal or sequence-to-sequence continual pretraining and allow selective freezing of blocks, insertion of new blocks, and logging of per-block metrics throughout training.

A practical track is to implement two scales:

- **Primary track:** a small model for fast ablations and seed runs.
- **Scale-validation track:** a larger model for one confirmation experiment after the method is stabilized.

This mirrors good continual-learning practice, where core claims are proven at manageable scale first and then checked for robustness at a second scale.[cite:65]

2. Architecture

The architecture should contain the following components:

- A stack of **frozen blocks** that store consolidated domain knowledge.
- One **current trainable block** that absorbs the active domain.

- A **selector/router** that combines outputs from all frozen blocks plus the current block.
- A **growth controller** that determines whether the current block is still learning effectively or whether a freeze-and-grow event is required.

The selector is important because once multiple blocks exist, a plain sequential stack makes retrieval implicit and entangled. An explicit selector lets the model learn which block is most relevant for each token or sequence position, rather than relying only on depth order.[cite:4]

3. Growth controller

The controller is the core novelty. It should monitor training dynamics and issue one of two decisions:

- **Continue training:** the current block still has useful spare capacity.
- **Freeze and grow:** the current block has saturated and a new block should be inserted.

A practical first version should use a **dual-signal criterion**:

- Smoothed probe or validation improvement signal.
- Loss-plateau or gradient-decay signal.

This choice is well aligned with the intended distinction from LLaMA Pro, which adds blocks by fixed design rather than from endogenous evidence.[cite:4] The controller should not be overcomplicated in the first paper. A thresholded dual-signal controller is easier to explain, reproduce, and ablate than a fully probabilistic decision rule.

4. Freeze-and-grow mechanism

When saturation fires, the current block should be frozen and a new block inserted above or after it. The recommended initialization is **warm-start cloning** of the current block, not zero initialization. That choice preserves recently learned structural priors and reduces the need to relearn shared features from scratch in the next domain.

The paper should still include a direct ablation against identity/zero-style initialization to make the comparison to LLaMA Pro explicit. The likely expected result is that identity/zero initialization is appropriate for one-shot injection, while warm-start is more efficient for sequential continual learning across related domains.[cite:4]

5. Routing and selector design

The first paper should avoid too many selector variants in the headline claim. The cleanest path is:

- Use one **default selector** for the main method.
- Include 2–3 lighter alternatives as ablations.

The preferred default is a **token-level selector** rather than a sequence-level gate. Token-level routing is conceptually stronger because different tokens in the same sequence may rely on different domains or blocks. Sequence-level gating can then be used as a simpler baseline to test whether token-level routing is actually worth the added complexity.

6. Replay and memory policy

Even though this paper is not centered on temporal drift, it is still a continual-learning paper and therefore needs a memory policy. The minimal credible replay design is:

- Store a small buffer from previously learned domains.
- Mix replay samples with current-domain samples during training on the active block.
- Clear or compact the buffer after freeze events according to the consolidation logic.

This makes the retention story measurable and defensible. It also ensures backward transfer is not interpreted only as architectural luck.

7. Training cycle

A full training run should proceed as follows:

1. Initialize the base model and designate the top block as the current trainable block.
2. Train on Domain 1 until the controller decides the block has learned enough or is saturated.
3. Freeze the current block, optionally store replay items, and insert a new trainable block.
4. Train on Domain 2 with replay from Domain 1.
5. Repeat the cycle across all domains.
6. After each domain, evaluate on all seen domains to compute continual-learning matrices and transfer metrics.

This protocol keeps the continual-learning interpretation intact and makes BWT central in the final analysis.[cite:65]

Recommended datasets

Primary recommendation: LLaMA Pro-style domain corpus

The current paper should use a **domain-sequential version of the LLaMA Pro setting**, because LLaMA Pro itself studies post-pretraining on code and math corpora and reports strong capability gains from block expansion.[cite:4] The best way to adapt this for continual learning is to split the new corpus into **domain episodes** rather than temporal periods.

Recommended domain order:

- General language or base-domain warmup.
- Code domain.
- Math domain.
- Mixed reasoning or instruction domain.

This preserves continuity with LLaMA Pro while making the problem truly continual: the model must learn one domain after another without forgetting earlier domains.[cite:4]

Dataset design principle

The paper does not need the exact original internal-scale corpus used by LLaMA Pro to be valid. What matters is preserving the same **task family**: code and mathematics as structured domains that stress compositional reasoning and specialized syntax.[cite:4] A student-feasible implementation can use open datasets from those domains and construct a controlled multi-domain stream.

Secondary benchmark options

Temporal benchmarks should not be the center of this paper, but they remain valuable as optional robustness checks because they are established for continual-learning evaluation:

- **TemporalWiki** uses differences between consecutive Wikipedia and Wikidata snapshots to evaluate retention of old knowledge and acquisition of updated knowledge.[cite:65][cite:63]
- Training on TemporalWiki diff data can achieve similar or better results than full-snapshot updating with much lower compute, which makes it especially useful for resource-aware follow-up studies.[cite:65][cite:67]
- **CC-News** is a large news corpus with naturally ordered updates and is available in curated versions such as the 708,241-article English subset spanning 2017–2019.[cite:66]

These two datasets are ideal for the next temporal paper, but for the current paper they should be presented as optional transfer tests rather than the main benchmark.[cite:65][cite:66]

Preferred benchmark structure

Main benchmark: domain-sequential continual pretraining

Construct a benchmark with 4–6 sequential domains. A strong version would include:

Phase	Domain	Purpose
D0	General or mixed warmup	Stabilize the base before specialization.
D1	Code	Test structured syntax and executable patterns.
D2	Mathematics	Test symbolic reasoning and formal composition.
D3	Mixed code-math or reasoning	Test cross-domain interference and transfer.
D4	Instruction or QA style data	Test generalization after specialization.

The value of this setup is that it turns LLaMA Pro's code-and-math adaptation idea into a continual-learning stream without abandoning the paper's original task family.[cite:4]

Optional benchmark: temporal robustness

A smaller TemporalWiki or CC-News experiment can be added as a final section to show that the same controller generalizes beyond domain sequences into evolving knowledge streams. [cite:65][cite:66] That is not required for the first submission, but it can strengthen the generality claim if compute allows.

Preferred baselines

The baseline suite should be compact but strong. Too many baselines dilute effort; too few weaken reviewer confidence.

Tier 1: mandatory baselines

Baseline	Why it matters
Naive sequential fine-tuning	Lower bound for forgetting in continual learning.
Replay-only	Separates memory benefit from architectural growth.
Fixed block expansion (LLaMA Pro-style)	Primary direct competitor and the paper to beat.[cite:4]
Architecture-only without replay	Tests whether structural isolation alone is sufficient.

These four baselines are enough to support the main claim if implemented cleanly.

Tier 2: strong modern additions

Baseline	Why it matters
LoRA or PEFT continual tuning	Efficient alternative that many reviewers will expect.
Prompt- or adapter-based continual method	Tests whether lightweight modularity can rival block growth.
Joint training upper bound	Not deployable, but useful as a performance ceiling.

These should be added if time permits, especially one PEFT baseline, because reviewers in 2026 will expect comparison against efficient adaptation approaches.

Baseline policy

The paper should treat **fixed-growth LLaMA Pro-style expansion** as the architectural baseline and **PEFT-style continual tuning** as the efficiency baseline. This creates a clean comparison triangle:

- Fine-tuning is simple but forgets.
- PEFT is efficient but may still interfere.
- Fixed block growth preserves more but wastes capacity.
- INCA grows only when needed and aims to preserve more while using capacity more intelligently.

Evaluation metrics

Core continual-learning metrics

The paper should report:

- **Average Accuracy (AA)** across all seen domains after each domain.[cite:65]
- **Backward Transfer (BWT)** to measure whether later learning harms or helps earlier domains.[cite:65]
- **Forward Transfer (FWT)** to measure whether prior learning helps future domains.[cite:65]

Backward transfer should remain central because the system is still a continual-learning architecture, and the strongest claim is not merely that it grows, but that it preserves earlier competence while learning later domains.

Capacity-specific metrics

Add 3 implementation-specific metrics:

- **Growth count:** how many blocks were added.
- **Domains per block:** how much knowledge each block absorbed before freezing.
- **Compute-normalized performance:** AA or BWT divided by training cost or by added parameters.

These metrics are essential because the paper's novel angle is not just accuracy, but **efficiency of capacity allocation**.

Optional selector metrics

If the selector is a substantial contribution, also report:

- Routing entropy.
- Fraction of tokens routed to each block.
- Block utilization diversity.

These help explain whether the selector is actually using historical blocks rather than ignoring them.

Main ablations

The ablation plan should be prioritized around the paper's central scientific questions.

A. Growth trigger ablation

Compare:

- Fixed growth every domain.
- Validation-improvement-only trigger.

- Loss-plateau-only trigger.
- Dual-signal trigger.

This is the single most important ablation because it directly validates the main novelty against LLaMA Pro's fixed schedule.[cite:4]

B. Initialization ablation

Compare:

- Identity/zero initialization.
- Warm-start clone.

This shows whether representational continuity matters for continual learning.

C. Routing ablation

Compare:

- Sequential stack without explicit selector.
- Sequence-level gate.
- Token-level selector.

This tests whether explicit retrieval improves retention and transfer.

D. Replay ablation

Compare:

- No replay.
- Small replay.
- Larger replay.
- Hard-only vs mixed replay sampling.

This identifies whether the architecture is doing the main work or whether replay dominates the result.

Experimental roadmap

Phase 1: proof of concept

- Use a small base model.
- Construct 4 domains.
- Run 3 seeds.
- Compare INCA against naive fine-tuning, replay-only, and fixed block expansion.

The goal is to prove that adaptive growth improves BWT or AA over fixed growth with similar or lower added capacity.

Phase 2: headline paper run

- Expand to 5–6 domains.
- Add a PEFT baseline.
- Add initialization and routing ablations.
- Report compute-normalized metrics.

The goal is to deliver a publishable continual-learning paper centered on intelligent capacity allocation.

Phase 3: optional extension

- Add one TemporalWiki or CC-News robustness experiment.
- Test whether the same controller generalizes to temporally evolving data.[cite:65][cite:66]

This phase is valuable but not necessary for the first submission.

Implementation priorities

What must be built first

1. Dynamic block freezing and insertion.
2. Continual evaluation matrix over domains.
3. Fixed-growth LLaMA Pro-style baseline.
4. Simple dual-signal controller.
5. Minimal replay buffer.

Without these, the core claim cannot be tested.

What can be delayed

- Hebbian updates.
- Advanced uncertainty-aware routing.
- Compression or merging of old blocks.
- Complex pruning before freeze.

These are better treated as later extensions once the central growth story works.

Recommended paper claim

The strongest publishable claim is:

A capacity-aware continual language model can outperform fixed block expansion by allocating new blocks only when saturation is detected, thereby improving retention and transfer with more efficient architectural growth.

This is more defensible than claiming a fully general continual-learning solution, and it is more novel than simply reproducing LLaMA Pro at smaller scale.[cite:4]

Final recommendation

The first paper should be framed as **capacity-aware continual block expansion for domain-sequential learning**, with LLaMA Pro as the primary architecture baseline and backward transfer as the core retention metric. LLaMA Pro demonstrates that block expansion is viable for code and math adaptation, while the proposed INCA direction should demonstrate that the **policy governing growth** is the true missing ingredient.[cite:4] TemporalWiki and CC-News should be retained as secondary or future-facing continual-learning benchmarks because they remain valuable for evaluating retention under evolving knowledge streams.[cite:65][cite:66]